LEARN BEING FORWARD

Programming — C Programming

File handling & standard library

COMPLETE LEARNING GUIDE

What You'll Learn

Real programs need to persist data beyond a single run and interact with the system they run on, and C's standard library provides exactly the primitives needed for both — the stdio.h file functions for reading and writing files, and a broader standard library (string.h, stdlib.h, math.h) covering string manipulation, memory allocation, conversions, and math operations that every C program relies on rather than reimplementing. File handling in C is deliberately low-level: you explicitly open a file into a stream, explicitly choose text or binary mode, explicitly check every operation for failure, and explicitly close the stream when finished — none of it happens automatically, which makes understanding the full open-use-check-close discipline essential rather than optional. This sub-module covers file I/O fundamentals, text vs. binary modes, error handling around file operations, a tour of the standard library's other core headers, and command-line arguments.

1. File I/O Basics: fopen, fclose, and Modes

fopen(filename, mode) opens a file and returns a FILE* stream handle, or NULL if the file couldn't be opened (doesn't exist, no permission, disk full, etc.) — checking for NULL before using the returned pointer is mandatory, not optional. The mode string controls both the operation ("r" read, "w" write — truncates any existing file, "a" append) and whether to add 'b' for binary mode ("rb", "wb"). Every successfully opened stream must eventually be closed with fclose(stream), which flushes any buffered writes to disk and releases the underlying OS file handle — forgetting to close a file can mean buffered data never actually reaches disk, not just a resource leak.

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("notes.txt", "w");
    if (fp == NULL) {
        perror("Failed to open notes.txt");
        return 1;
    }

    fprintf(fp, "Line 1\n");
    fprintf(fp, "Line 2\n");

    fclose(fp); // flushes buffered writes and releases the file handle
    return 0;
```

```
}
```

2. Reading and Writing: Formatted, Line-Based, and Block I/O

fprintf/fscanf work like printf/scanf but target a file stream instead of standard output/input, suited to structured text data. fgets(buffer, size, stream) reads one line (up to size-1 characters, stopping at a newline) into buffer and is the safe, bounds-respecting way to read text lines — unlike the deprecated gets(), which has no way to limit how much it reads and is a classic buffer-overflow vector. fread/fwrite transfer raw blocks of bytes (often an entire struct at once) and are the standard tool for binary file I/O, taking the data pointer, the size of one element, the number of elements, and the stream.

- Never use gets() — it has no bounds argument at all and was formally removed from the C11 standard for this reason; use fgets(buf, sizeof(buf), stdin) instead.
- fgets keeps the trailing newline character in the buffer (unlike scanf's %s) — strip it manually if it isn't wanted.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *fp = fopen("notes.txt", "r");
    if (fp == NULL) { perror("open failed"); return 1; }

    char line[100];
    while (fgets(line, sizeof(line), fp) != NULL) {
        line[strcspn(line, "\n")] = '\0'; // strip the trailing newline
        printf("Read: %s\n", line);
    }

    fclose(fp);
    return 0;
}
```

3. Error Handling in File Operations

Every file operation can fail, and C expects the programmer to check for failure explicitly rather than relying on exceptions. fopen returning NULL signals it couldn't open the file at all; feof(stream) reports whether the end of the file has been reached (but only AFTER a read has already attempted to go past the end, not before); ferror(stream) reports whether a genuine read/write error occurred, distinct from simply reaching the end of the file. perror(message) prints a human-readable description of the last system error alongside your own message, which is far more useful for debugging than a generic failure print.

- Checking feof() BEFORE attempting a read (while (!feof(fp))) is a classic bug — it can cause one extra, spurious iteration. Check the read function's own return value instead (while (fgets(...) != NULL)).

- perror("context message") appends the OS's actual reason (e.g., 'No such file or directory') automatically, from the errno set by the failed call.

4. Binary vs. Text File Modes

Text mode ("r"/"w") may translate line-ending characters on some platforms (notably, Windows translates between \n and \r\n), which is invisible and harmless for plain text but corrupts binary data if applied to it. Binary mode ("rb"/"wb") disables that translation and transfers bytes exactly as they are, which is mandatory for images, serialized structs, or any data where byte-for-byte fidelity matters. On POSIX systems (Linux/macOS) there's no actual difference between the two modes since no translation happens either way, but writing code as if the distinction always matters keeps it portable to Windows.

```c
#include <stdio.h>

struct Record { int id; double value; };

int main(void) {
    struct Record r = {1, 3.14};

    FILE *fp = fopen("data.bin", "wb"); // binary mode — no text translation
    if (fp) {
        fwrite(&r, sizeof(struct Record), 1, fp);
        fclose(fp);
    }

    struct Record loaded;
    fp = fopen("data.bin", "rb");
    if (fp) {
        fread(&loaded, sizeof(struct Record), 1, fp);
        fclose(fp);
        printf("id=%d value=%.2f\n", loaded.id, loaded.value);
    }
    return 0;
}
```

5. Touring the C Standard Library

Beyond stdio.h, three headers cover most everyday needs: string.h provides strlen/strcpy/strcmp/strcat and their bounded variants for text manipulation; stdlib.h provides malloc/calloc/realloc/free for dynamic memory, atoi/atof/strtol for parsing numbers out of strings, and qsort/rand for sorting and pseudo-random numbers; math.h provides floating-point functions like sqrt, pow, floor, ceil, and the trigonometric functions, and historically required linking with -lm on Linux/macOS (gcc program.c -lm) since the math implementation sometimes lives in a separate library file from the rest of the standard library.

- atoi("42") returns 42 (int) but silently returns 0 on invalid input with no way to detect the error — strtol is the safer, error-checkable alternative.
- qsort(array, count, elementSize, comparatorFunction) sorts any array type via a function pointer comparator, the same pattern covered in the Control Flow sub-module.

6. Command-Line Arguments: argc and argv

A C program can receive arguments typed on the command line by declaring main with the signature int main(int argc, char *argv[]) — argc holds the count of arguments including the program's own name, and argv is an array of C strings holding each argument's text, with argv[0] always being the program's invocation name and argv[argc] guaranteed to be NULL. Numeric arguments arrive as text and must be explicitly converted (with atoi, strtol, or atof) before use in arithmetic — a very common beginner mistake is trying to add argv[1] + argv[2] directly, which adds pointer addresses rather than the numbers they represent as text.

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <num1> <num2>\n", argv[0]);
        return 1;
    }

    int a = atoi(argv[1]); // must convert text -> int before arithmetic
    int b = atoi(argv[2]);
    printf("Sum = %d\n", a + b);
    return 0;
}
/* Run as: ./program 3 4   ->  Sum = 7 */
```

Common Pitfalls

- Not checking fopen's return value for NULL before using the stream, causing a crash on a missing or inaccessible file.
- Using the deprecated, unbounded gets() instead of fgets() with an explicit buffer size.
- Checking feof() before attempting a read instead of checking the read function's own return value, causing a spurious extra iteration.
- Opening binary data (images, structs) in text mode, risking silent byte corruption from newline translation on some platforms.
- Forgetting to fclose() an opened file, risking buffered writes never being flushed to disk.
- Treating argv elements as numbers directly in arithmetic without first converting them from text with atoi/strtol/atof.

- Using atoi for input validation when it silently returns 0 for invalid text instead of signaling an error — strtol with error checking is safer.

Key Takeaways

- Always check fopen's return value for NULL, and always pair a successful fopen with a matching fclose.
- fgets with an explicit size is the safe replacement for the removed, unbounded gets().
- Check a read operation's own return value to detect end-of-file/errors — don't rely on feof() checked before the read.
- Binary mode ("rb"/"wb") prevents newline-translation corruption of non-text data; always use it for struct/image/raw-byte I/O.
- stdlib.h's atoi/strtol/atof convert command-line text arguments into usable numeric types before arithmetic.
- main(int argc, char *argv[]) is the standard way to receive command-line arguments, with argv[0] always the program name.

Learn More

cppreference — C File I/O
https://en.cppreference.com/w/c/io

GeeksforGeeks — File Handling in C
https://www.geeksforgeeks.org/c/basics-file-handling-c/